Final Project Report

AI Manufacturing Platform

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1.	Aaditya Shrivastava	EN22CS301007
2.	Aakansha Jaiswal	EN22CS301011
3.	Akanksha Sharma	EN22CS301080
4.	Akshita Bhawsar	EN22CS301098

Group Name: Group 12D1

Project Number: DO-12

Industry Mentor Name: Aashruti Shah

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-2026

Problem Statement & Objectives

1. Problem Statement

The deployment and management of modern manufacturing applications require scalable, reliable, and highly available infrastructure. Traditional deployment methods, which rely on manual configuration and static environments, often lead to issues such as downtime during updates, inconsistent environments, and difficulty in scaling applications based on demand.

In the context of a manufacturing application developed using Python ensuring seamless deployment, efficient resource utilization, and continuous availability becomes a critical challenge. Manual deployment processes are error-prone and do not support automated updates making them unsuitable for production-grade systems.

Furthermore, maintaining zero downtime during application updates is essential for ensuring uninterrupted service to end-users. However, achieving this using conventional deployment strategies is complex and lacks automation.

Therefore, there is a need to design and implement a robust DevOps-based solution that enables containerization, automated deployment, and orchestration of the manufacturing application using modern tools. The system should support scalable deployment on a Kubernetes cluster, Docker , GitHub (Github Action) efficient workload management and zero-downtime updates through rolling deployment strategies.

2. Project Objectives

The primary objective of this project is to deploy and manage a Manufacturing Python Flask application using modern DevOps practices and Kubernetes orchestration.

The specific objectives are as follows:

1. To containerize the Manufacturing application using Docker, ensuring consistency across development, testing and production environments.
2. To deploy the application on a Kubernetes cluster (Minikube) for efficient orchestration and management of containerized workloads.
3. To design and implement Kubernetes deployment manifests, defining pods, replicas and configurations for scalable and reliable application deployment.
4. To create Kubernetes service manifests to expose the application and enable communication between components and end-users.

5. To automate the build and deployment pipeline using tools like Git and CI/CD platforms (GitHub Actions).
6. To ensure high availability and scalability, allowing the system to handle varying workloads efficiently.
7. To enable version control and continuous integration, ensuring faster development cycles and reliable releases.
8. To maintain system reliability and fault tolerance, using Kubernetes features such as self-healing and auto-restart of containers.
9. To demonstrate real-world DevOps practices, bridging the gap between development and operations in a production-like environment.

Scope of the Project

The scope of this project focuses on the design, containerization, deployment, and orchestration of a Manufacturing Python Flask application using modern DevOps practices and Kubernetes infrastructure.

The project primarily covers the following aspects:

1. Application Containerization

- Packaging the Manufacturing python application into Docker containers
- Ensuring consistency across development, testing and deployment environments
- Managing dependencies within containerized environments

2. Kubernetes-Based Deployment

- Deploying the application on a Kubernetes cluster using Minikube
- Creating and configuring Deployment manifests for managing pods and replicas
- Ensuring scalable and efficient workload management

3. Service Configuration

- Defining Kubernetes Services to expose the application
- Enabling communication between application components and external users

- Managing networking within the cluster

4. CI/CD Integration

- Implementing automated build and deployment pipelines using Git and CI/CD tools (GitHub Actions)
- Automating Docker image creation and deployment updates
- Ensuring faster and reliable release cycles

5. Scalability and High Availability

- Utilizing Kubernetes features like auto-scaling and load balancing
- Ensuring the application remains available under varying workloads

Out of Scope

The project does not include:

- Development of complex manufacturing algorithms or business logic
- Integration with real industrial systems (IoT)
- Advanced cloud deployment
- Multi-user authentication and role-based access control
- Large-scale distributed system optimization

Proposed Solution

To address the challenges of deploying and managing a manufacturing application efficiently, this project proposes a DevOps-driven, containerized and orchestrated solution using modern tools such as Docker, Kubernetes and CI/CD pipelines.

The solution focuses on enabling reliable deployment, environment consistency, and basic scalability of a Python Flask-based Manufacturing Application by leveraging containerization and orchestration techniques.

1. Key features

The proposed DevOps-based deployment system offers several important features that ensure efficient, reliable and scalable management of the Manufacturing application:

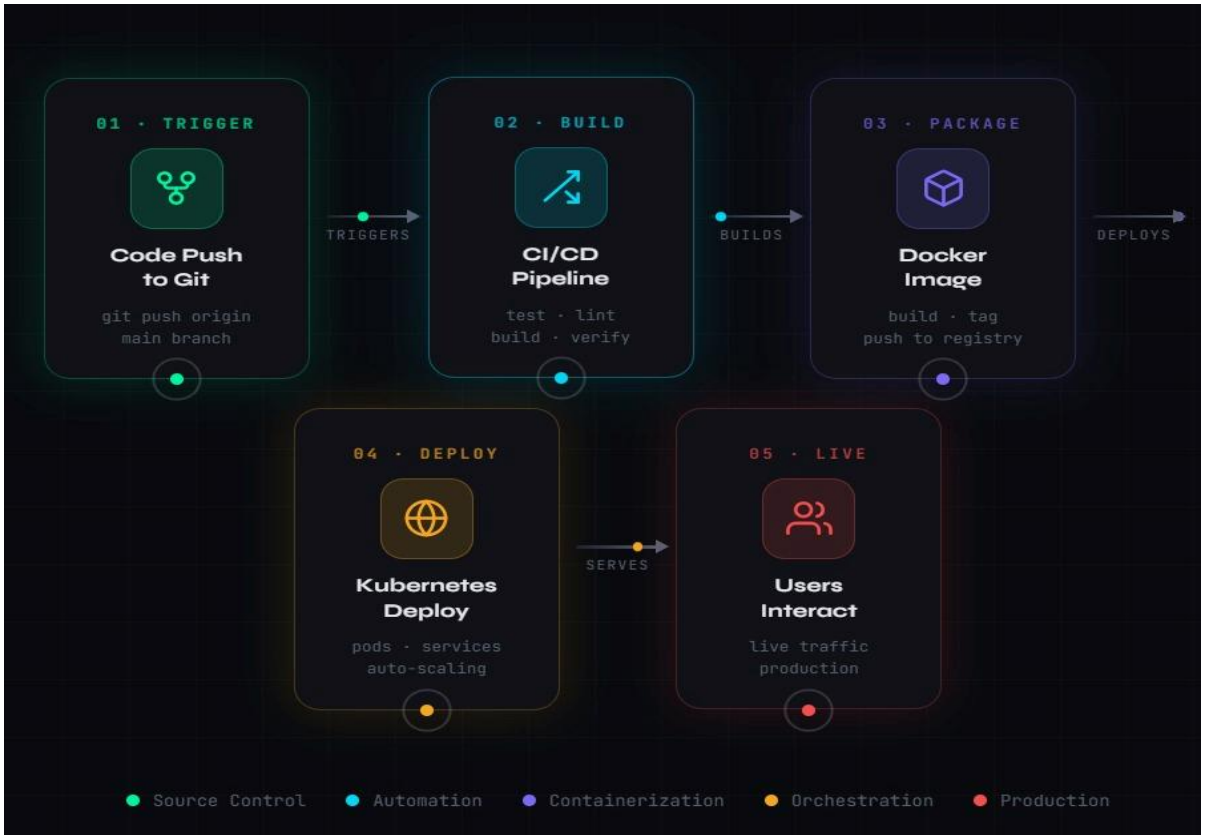
- **Containerization using Docker**
  The application is packaged into Docker containers, ensuring a consistent

runtime environment across development, testing and deployment stages. This eliminates dependency-related issues and improves portability.

- **Kubernetes-Based Deployment**
  The application is deployed on a Kubernetes cluster (Minikube) which manages container orchestration and lifecycle management efficiently.
- **Scalability through Replication**
  Kubernetes allows the application to run multiple replicas, enabling the system to handle increased load and ensuring better availability.
- **Service Exposure and Networking**
  Kubernetes Services are used to expose the application, allowing seamless communication between the application and end-users while managing internal cluster networking.
- **CI/CD Pipeline Integration**
  Automated pipelines using Git and tools like GitHub Actions. streamline the process of building, testing, and deploying the application, reducing manual effort and errors.
- **Self-Healing Capability**
  Kubernetes automatically detects failures and restarts unhealthy containers, ensuring continuous application availability.
- **Environment Consistency**
  Docker ensures that the application behaves the same way across all environments, reducing deployment-related inconsistencies.
- **Portability**
  The containerized application can run on different systems and platforms without modification, making it flexible for future deployments.
- **Simplified Deployment Management**
  Kubernetes manifests (Deployment and Service files) provide a structured and declarative way to manage application deployment.
- **Real-World DevOps Implementation**
  The project demonstrates practical usage of DevOps tools and practices bridging the gap between development and operations.

2. Overall Architecture and Workflow

The overall architecture of the AI Manufacturing Platform is designed using a layered, modular and DevOps-driven approach, integrating application development, AI processing and deployment automation within a unified system.



The architecture consists of the following major components:

Development Layer

This layer represents the starting point of the system:

- Developers write and update the application code
- Source code is managed using Git (Version Control System)
- Changes are pushed to a centralized repository

This ensures proper version tracking, collaboration and code management.

CI/CD Pipeline Layer

The CI/CD pipeline automates the build and deployment process:

- GitHub Actions Tools is used
- On every code push:
    - Code is built and tested
    - Docker images are created

- Infrastructure configurations are managed using Infrastructure as Code (IaC)

This layer ensures automation, faster delivery, and reduced manual errors.

Containerization Layer

- The application is packaged into Docker containers
- Containers include all dependencies and runtime configurations
- This ensures portability and consistency across environments

Kubernetes Cluster Layer

This is the core execution environment of the system:

- The containerized application is deployed on a Kubernetes Cluster
- The cluster manages:
  - Pod creation and lifecycle
  - Scaling and load balancing
  - Application availability

Inside the cluster, the AI system consists of:

- Language Model Module → Generates structured content
- Image Generation Module → Produces visual prototypes
- Multi-Agent System → Performs research and report generation

Deployment Environment

The application can be deployed in:

- Local Environment (Minikube) → for development/testing
- Cloud Environment (AWS) → for scalable production deployment

Overall Workflow

The system follows a structured DevOps and AI processing workflow:

1. **Code Development**
   - Developers write code and push it to the Git repository
2. **CI/CD Trigger**
   - Code push triggers the CI/CD pipeline automatically

3. **Build and Containerization**
    - Application is built
    - Docker image is created and store.

4. **Deployment to Kubernetes**
    - Kubernetes pulls the latest Docker image
    - Deployment manifests create and manage pods
5. **Application Execution**
    - The application runs inside the Kubernetes cluster
    - AI modules process user requests:
        - Language model generates text
        - Image model generates visuals
        - Multi-agent system creates reports
6. **User Interaction**
    - Users interact with the system through the frontend
    - Requests are processed by backend services
7. **Monitoring and Logging**
    - System performance and logs are continuously monitored

3. Tools & Technologies Used

The AI Manufacturing Platform is developed and deployed using a combination of modern tools and technologies across development, AI processing, and DevOps practices. These tools ensure scalability, automation, and efficient system performance.

1. Programming Language

Python: Used as the primary programming language for backend development due to its simplicity, extensive libraries, and strong support for AI/ML integration.

2. Frontend Technology

Streamlit : Used to build an interactive and user-friendly web interface that allows users to input manufacturing queries and view generated outputs such as text, images, and reports.

3. AI Technologies

- **Large Language Models (LLMs)**: Used for generating structured technical descriptions and reports.
- **Image Generation APIs**: Used to create visual prototypes from textual inputs.
- **Multi-Agent System (CrewAI)**: Enables collaboration between agents such as Research Agent and Writer Agent to perform complex workflows.

4. Version Control

Git: Used for source code management, version tracking, and collaboration among team members.

5. Containerization

Docker: Used to package the application and its dependencies into containers, ensuring consistency across environments.

6. Orchestration

Kubernetes (Minikube): Used to deploy, manage, and scale containerized applications. Minikube provides a local Kubernetes environment for testing and development. (Also we used instances type: m7i-flex.large)

7. CI/CD Tools

GitHub Actions: Used to automate build, testing, and deployment processes, enabling continuous integration and delivery.

8. Development Tools

Visual Studio Code (VS Code): Used as the primary IDE for coding, debugging, and project development

Results & Output

The AI Manufacturing Platform was successfully implemented and deployed using Docker and Kubernetes, demonstrating a complete DevOps workflow from development to deployment. The system was tested with multiple manufacturing-related inputs and produced consistent, reliable outputs.

1. System Interface and Outputs

Figure 1: Main User Interface

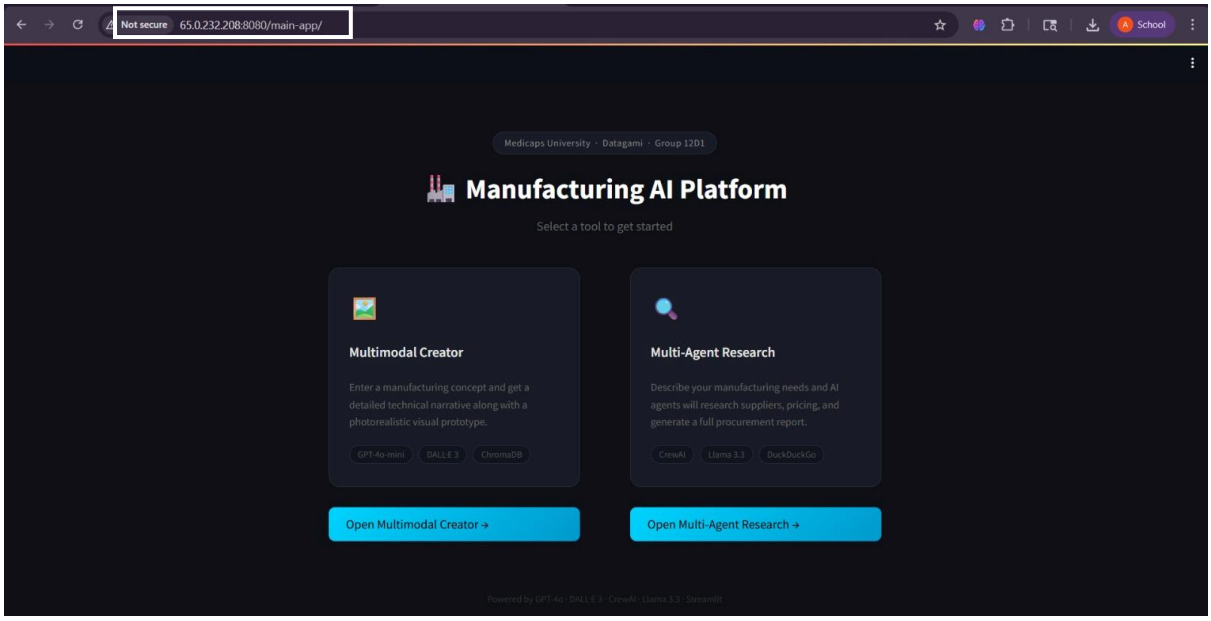


Figure 2:Multi-Agent User Interface

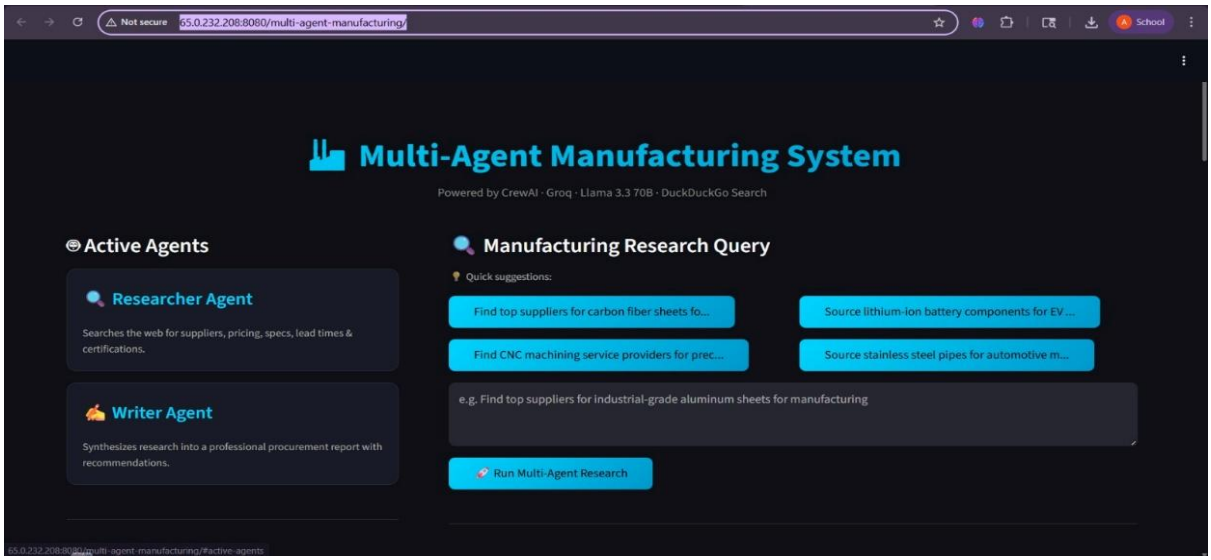


Figure 3: Multimodal Creator User Interface

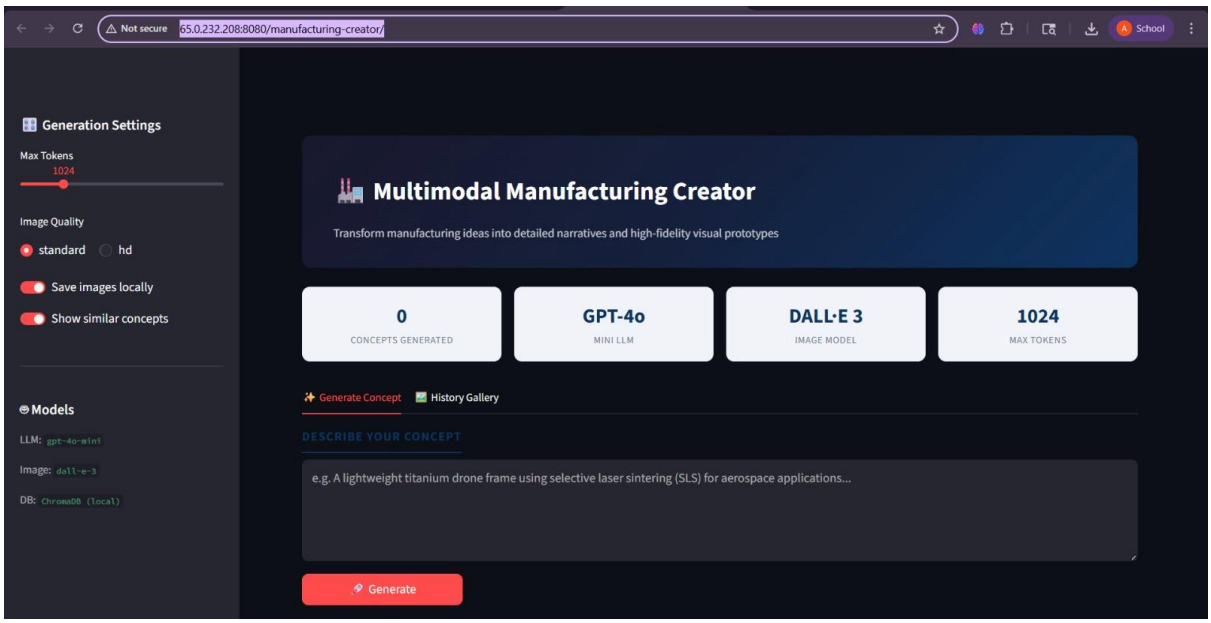


Figure 4: GitHub Action

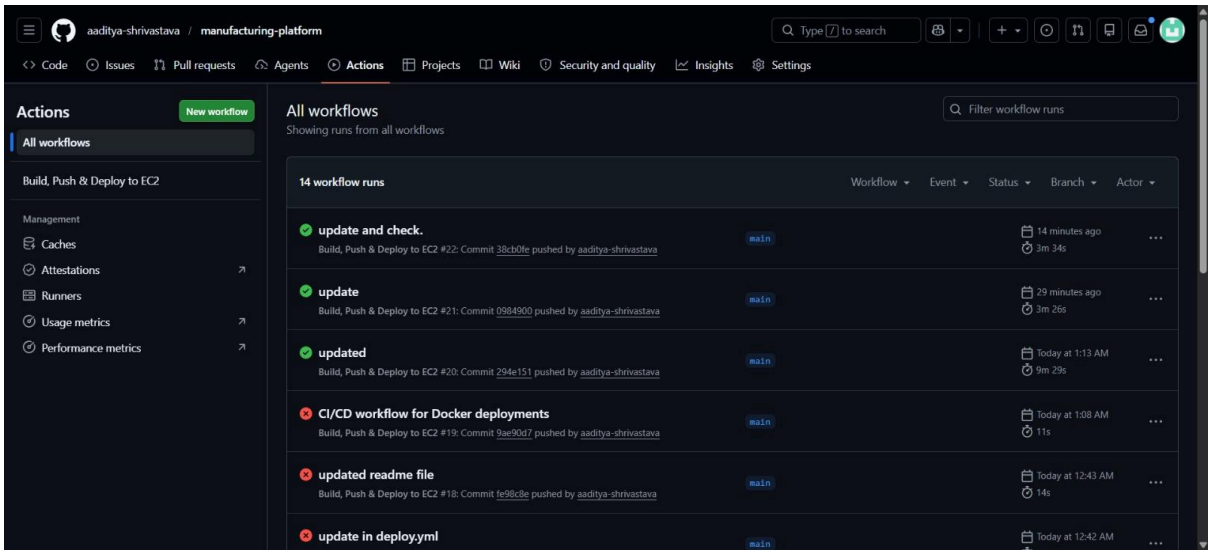
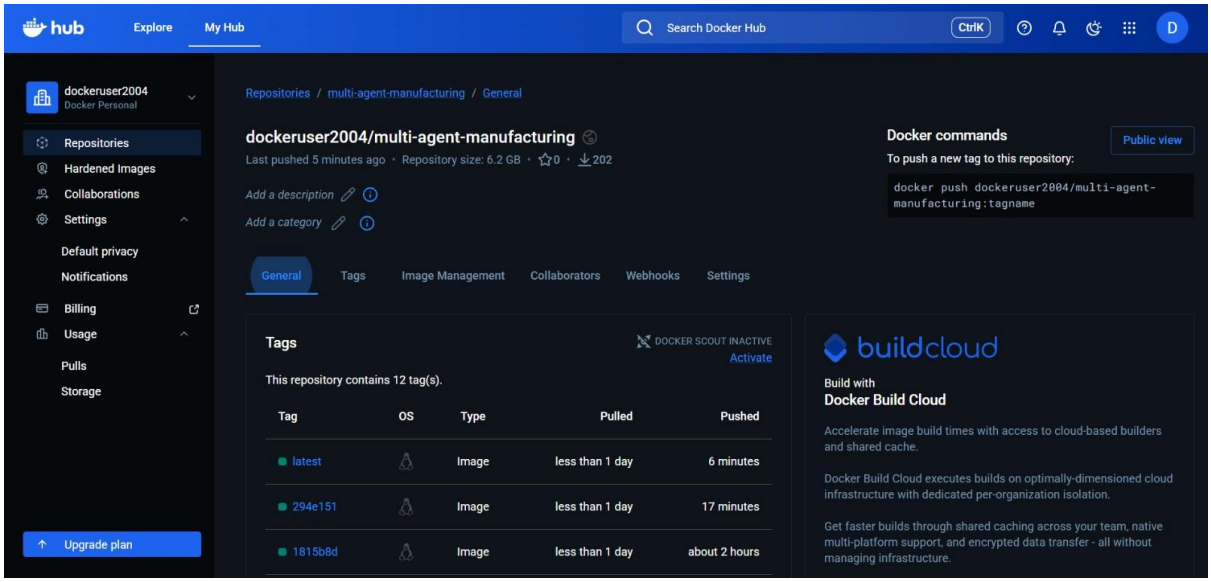


Figure 5: Kubernetes Pods run on EC2

Figure 6: Docker



2. Key Outcomes

The successful implementation of the AI Manufacturing Platform and its deployment using DevOps practices resulted in the following key outcomes:

1. **Successful Containerized Deployment**
   The Manufacturing application was effectively containerized using Docker and deployed on a Kubernetes (Minikube) cluster, ensuring consistency across environments.

2. **End-to-End DevOps Workflow Implementation**
   The project demonstrated a complete DevOps pipeline, including code management (Git), containerization (Docker), and orchestration (Kubernetes).

3. **Automated Build and Deployment Process**
   CI/CD tool (GitHub Actions) enabled automation of building Docker images and updating deployments, reducing manual effort and errors.

4. **Scalable and Reliable Application Deployment**
   Kubernetes ensured scalability through replica management and improved reliability through self-healing capabilities.

5. **Efficient Resource Management**
   The system efficiently managed application workloads using Kubernetes pods and services.

6. **Functional AI-Based Output Generation**
   The platform successfully generated structured technical content, reports and outputs based on user inputs.

7. **Improved Application Availability**
   The use of multiple pods ensured that the application remained available even in case of individual container failures.

8. **Real-World DevOps Exposure**
   The project provided hands-on experience with industry-relevant tools and

9. **Foundation for Future Enhancements**
   The project establishes a base for further improvements such as cloud deployment, advanced scaling, monitoring tools and enhanced AI capabilities

Conclusion

The AI Manufacturing Platform successfully demonstrates the integration of modern DevOps practices with an intelligent application workflow. The project achieved its primary goal of deploying a Manufacturing application using Docker containerization and Kubernetes orchestration, ensuring a reliable and consistent deployment environment.

Through this project, a complete end-to-end DevOps pipeline was implemented, starting from code management using Git, followed by automated build processes using CI/CD tools, and culminating in deployment on a Kubernetes cluster. This significantly reduced manual effort, improved deployment efficiency and ensured consistency across environments.

The use of Kubernetes enabled effective management of application workloads, providing features such as replica-based scalability, self-healing, and service exposure, which contributed to improved system reliability and availability. The system was able to handle multiple requests efficiently while maintaining stable performance.

Additionally, the integration of AI capabilities within the application enhanced its functionality by enabling automated generation of structured outputs, making the platform more intelligent and useful for manufacturing-related tasks.

Overall, the project highlights the importance of combining DevOps methodologies with modern application development, demonstrating how automation, containerization, and orchestration can improve the deployment and management of real-world applications.

In conclusion, this project serves as a strong foundation for building scalable, reliable, and intelligent systems and provides valuable practical experience in implementing industry-relevant DevOps tools and technologies.

Future Scope & Enhancements

The AI Manufacturing Platform provides a strong foundation for integrating DevOps practices with intelligent application workflows. However, there are several opportunities to enhance the system further and extend its capabilities for real-world, production-level use.

1. Cloud-Based Deployment

- Deploy the application on cloud platforms such as Azure or GCP
- Enable high availability, global access and better scalability
- Use managed Kubernetes services (EKS, AKS, GKE)

2. Advanced CI/CD Pipeline

- Implement fully automated pipelines with testing, security scans and deployment stages
- Add rollback mechanisms for safer deployments
- Integrate tools like SonarQube for code quality analysis

3. Monitoring and Logging Integration

- Integrate tools like Prometheus and Grafana for real-time monitoring
- Use ELK Stack (Elasticsearch, Logstash, Kibana) for centralized logging
- Enable alerting mechanisms for system failures

4. Auto-Scaling Implementation

- Implement Kubernetes Horizontal Pod Autoscaler (HPA)
- Automatically scale application pods based on CPU/memory usage or traffic

5. Enhanced Security Features

- Implement authentication and authorization (RBAC)
- Secure API communication using HTTPS and token-based authentication
- Use Kubernetes Secrets and Vault for secure credential management

6. Microservices Architecture

- Convert the application into independent microservices
- Improve scalability, flexibility, and maintainability
- Enable independent deployment of modules

7. Advanced AI Capabilities

- Improve AI models for better accuracy and performance
- Add more specialized agents (cost analysis, quality control, supply chain optimization)
- Integrate real-time data sources for dynamic insights

8. User Management System

- Add multi-user support with login/signup functionality
- Implement role-based access control (Admin/User roles)

9. Integration with Industrial Systems

- Connect with ERP systems and manufacturing databases
- Enable real-world industrial data processing
- Integrate IoT devices for real-time manufacturing insights

10. Performance Optimization

- Optimize resource usage and reduce latency
- Implement caching mechanisms for faster responses
- Improve API efficiency and response time